Project Report

Build an algorithm to create an I/O identity for an application.

CS550 – Advanced Operating Systems

May 01, 2023

• Payal Bedare A20525785
• Abhishek Kumbhar A20525815

Under the guidance of Neeraj Rajesh

Introduction:

When optimizing parallel applications at scale, we often focus on computation-communication aspects and I/O often gets limited attention. The utilization and performance of storage, computing, and network resources within HPC data centers have been studied extensively, but with the increasing performance gap between compute and I/O subsystems, improving I/O performance remains one of the major challenges [1]. So, to make an I/O identity from already existing data and clustering, we used some clustering algorithms, and their tradeoffs. By having I/O Identity we can use it to classify I/O applications so that optimization found for one application in a class can be attempted to the entire class.

This research attempts to increase our understanding of I/O patterns in distributed contexts by categorizing the I/O of an application. The purpose of this project is to provide a framework for categorizing the I/O of an application, which allows information to be transferred across apps and enhance the performance and understanding of distributed systems.

Background:

As the parallel file system is a shared resource, few jobs running on a system can significantly impact the performance of other applications. Darshan [2] is an I/O characterization tool developed at Argonne National Lab. It helps to get an accurate picture of application I/O by showing access patterns, sizes, number of operations, etc. with minimum overhead. Darshan can utilize portable, low-overhead mechanisms for intercepting I/O routines.

Also, [3] Clustering is a Machine Learning technique that involves the grouping of data points. Given a set of data points, we can use a clustering algorithm to classify each data point into a specific group. In theory, data points that are in the same group should have similar properties and/or I/O features, while data points in different groups should have highly dissimilar properties and/or I/O features. Clustering is a method of unsupervised learning and is a common technique for statistical data analysis used in many fields. So, we used clustering analysis to gain some valuable insights from our data by seeing what groups the data points fall into when we apply a clustering algorithm.

Motivation:

The traditional approach of manually assigning I/O identities to each application can be time-consuming, error-prone, and may not scale well. To overcome these challenges, we propose to develop an algorithm in which we can group applications based on their similarities, such as the type of data they process, the protocols they use, and their I/O requirements. Once the applications are grouped into clusters, we can assign unique I/O identities to each cluster, which can then be used for all the applications within that cluster.

To implement the clustering algorithm, we plan to use a combination of Python and machine learning libraries such as scikit-learn. We will train the algorithm on historical application data from Argonne National Lab, which will allow the system to adapt to changes in the framework.

Implementation:

In this project, we aimed to cluster I/O properties based on their performance characteristics. The dataset used in this project is from Argonne National Laboratory. The dataset contains various performance metrics related to I/O operations of several applications.
Firstly, we loaded the dataset and removed any null values and duplicate records. Then, we dropped some columns that were not relevant to our analysis.

After data cleaning, we performed outlier removal using the Interquartile Range (IQR) method. Any records that had any values outside of the 1.5*IQR ranges were considered as outliers

and removed from the dataset.The data is then standardized using the StandardScaler from scikit-learn, which scales each column of the data to have zero mean and unit variance.

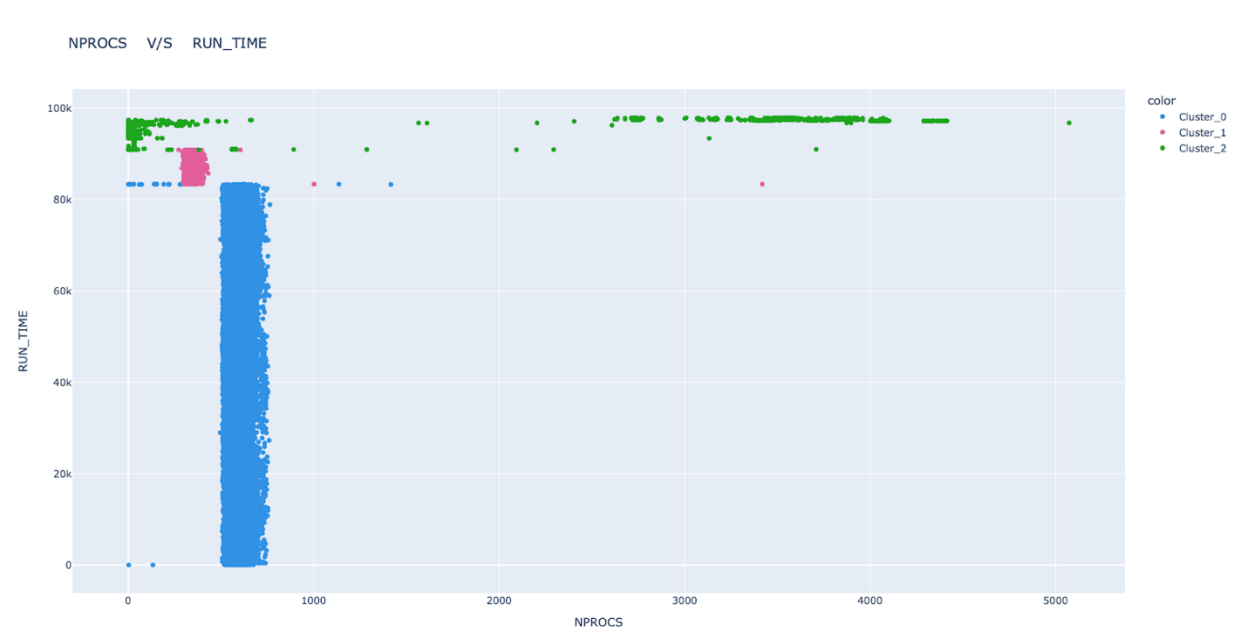
Next, we performed clustering on the remaining data using the KMeans & DBSCAN algorithm. We set the number of clusters to 3 by the analysis from the elbow method for finding the optimal number of clusters.

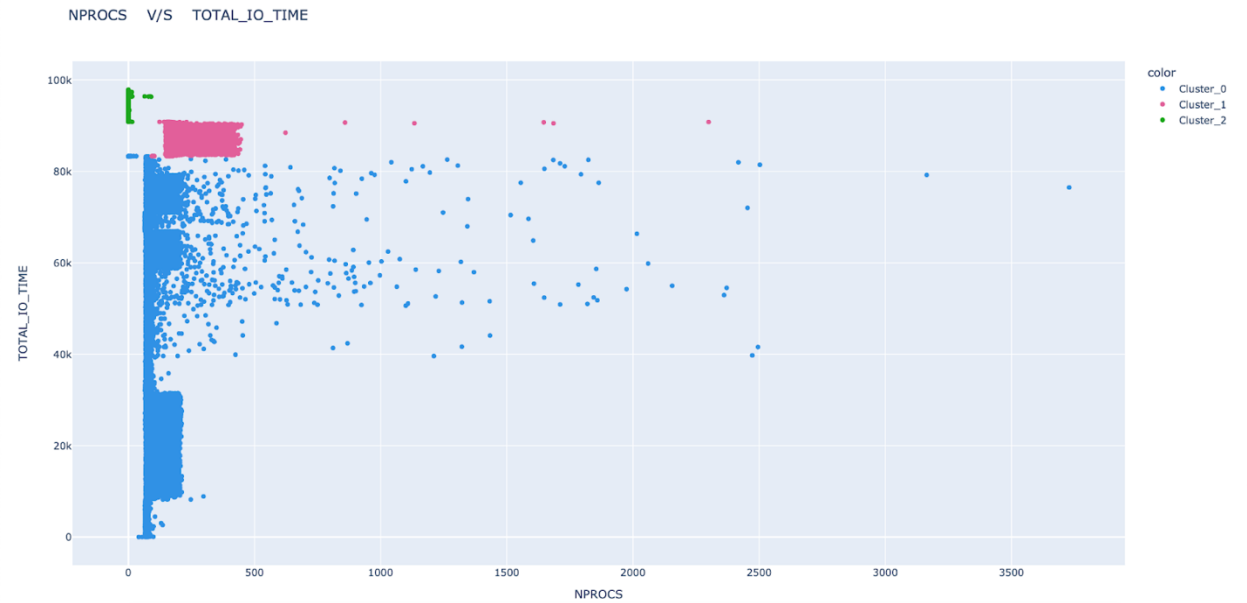
In the DBSCAN algorithm, on the input data, using a grid search over different combinations of hyperparameters. The goal is to find the combination of hyperparameters that yields the best Silhouette score, which is a measure of how well the data points within each cluster are separated from each other.Next, a grid search is performed over a range of values for the two hyperparameters of DBSCAN: epsilon and min_samples. The grid search generates all possible combinations of these hyperparameters, and for each combination, DBSCAN is applied to the data. If the resulting number of clusters is less than 2 or greater than 50, the combination is discarded. For all other combinations, the Silhouette score is calculated and stored.

The combination of hyperparameters that yields the highest Silhouette score is then selected, also optimizing the hyperparameters using a grid search to find the best possible clustering solution.

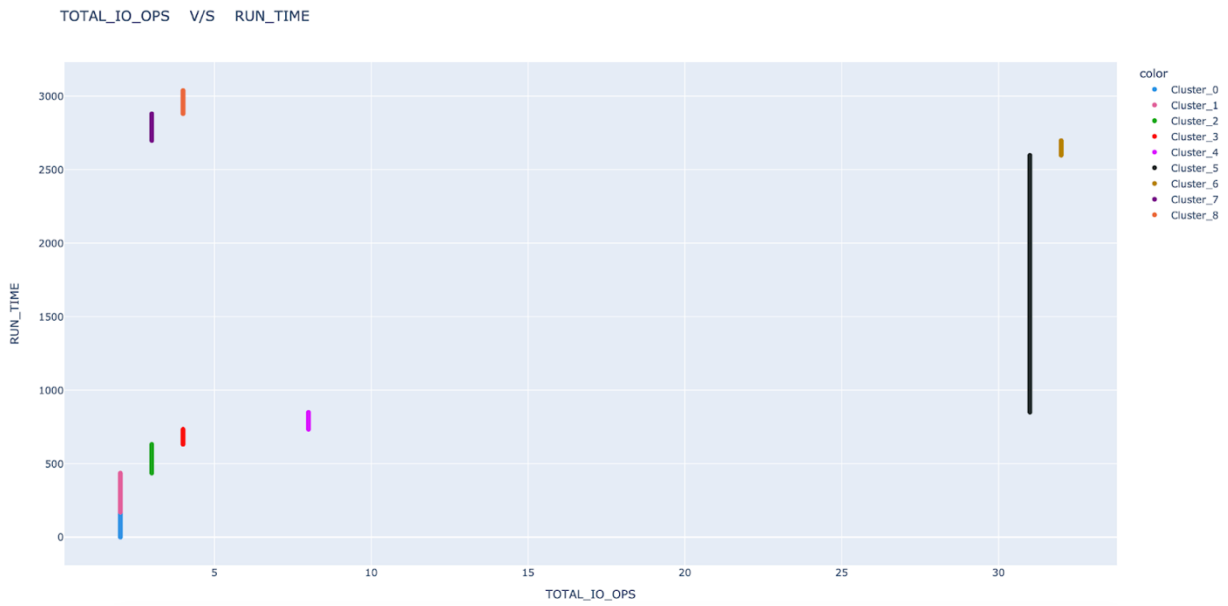
Results:

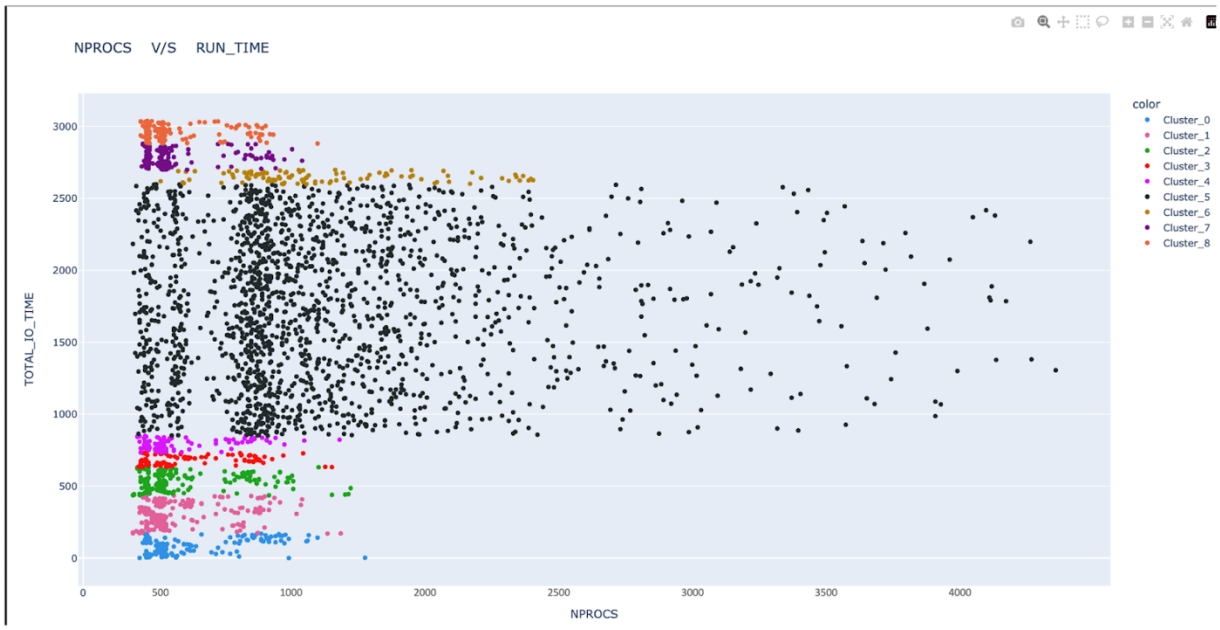
1] K-Means Clustering on given dataset





2] DBSCAN Results on given dataset





Conclusion:

In conclusion, the analysis of the NPROCS and TOTAL_IO_TIME data revealed that the K-Means algorithm identified three clusters, whereas DBSCAN identified nine clusters. The scatterplots obtained from the K-Means algorithm demonstrated that most of the data points were concentrated in cluster-0, with limited dispersion of data points. In contrast, DBSCAN showed a distinct scattering of data points, particularly in cluster-5, resulting in more evenly spread and well-defined clusters. Therefore, it is evident that DBSCAN produced more distinct and clear clusters than K-Means algorithm, indicating that DBSCAN may be a better choice for clustering this particular data set.

Therefore, our work has introduced a methodology for characterizing I/O workloads in a continuous, and scalable manner. By demonstrating the value of this approach in both understanding system behavior and accelerating debugging of individual applications, we have shown how it can enable more efficient and effective data-intensive computing.

References:

[1] Glenn K. Lockwood; Shane Snyder; Suren Byna; Philip Carns; Nicholas J. Wright, “Understanding Data Motion in the Modern HPC Data Center.”

[2] Pramod Kumbhar, “I/O performance analysis with Darshan” 2020/03

[3] Jiang Zhoua, Yong Chena, Dong Daia, Yu Zhuang “I/O Characteristic Discovery for Storage
System Optimizations”